

Auditable Semantic Partitioning with Local Language Models

Anthony Paterson

Fractal Media Infrastructure

Gracemere, Queensland, Australia

fractalmediainfrastructure@gmail.com

Author manuscript preprint, v0.1

Not peer reviewed.

Auditable Semantic Partitioning with Local Language Models

Fixed Bucket Budgets, Overflow, and Reviewable Data Work

Abstract

Many data-work tasks require people to convert messy records into workable categories: interview excerpts into themes, software issues into repair lanes, research notes into analytic clusters, and support records into operational failure modes. Existing approaches include human taxonomies, supervised classification, topic modeling, embedding clustering, and human-in-the-loop annotation. This paper proposes auditable semantic partitioning as a complementary practice for exploratory categorization with local language models. In this workflow, the analyst declares a dataset, sorting intent, field projection, model, and positive bucket count. The model proposes a semantic partition within that fixed budget. The runtime freezes the plan, enforces exactly that number of positive buckets plus an overflow bucket, and records preflight judgments, bucket definitions, assignments, confidence scores, rationales, review flags, and analyst notes. A run is treated not as a final truth claim but as a reviewable semantic probe. Comparing runs across bucket counts, models, field projections, and bucket-generation modes can reveal stable anchors, model-prior effects, weak-fit items, and projection-sensitive meanings. Drawing on two anonymized local-first semantic-sorting projects, the paper defines bucket count as semantic bandwidth, junk/overflow as epistemic restraint, rationales as audit depositions, and semantic triangulation as repeated comparison under declared conditions. It argues that the value of local model-assisted sorting lies less in automatic labeling than in making interpretive compression visible, inspectable, and contestable.

Keywords: Big Data practices; semantic partitioning; language models; data work; local AI; auditability; human-in-the-loop; topic modeling; semantic compression; model comparison

1. Introduction

Categorization is one of the ordinary labors of data work. A researcher codes interview fragments. A maintainer sorts issue reports. A teacher groups student questions. A policy analyst separates consultation responses. A project owner turns a directory of notes into operational lanes. In each case, the surface action is sorting, but the underlying work is interpretive: deciding what structure is useful, what the categories are for, and what to do with items that do not fit.

Contemporary language models make this work tempting to automate. A user can paste records into a model and ask it to group them. The result may look persuasive: category names, item assignments, and fluent explanations. Yet the apparent neatness can hide several weaknesses. The model may invent a taxonomy that drifts across the run. It may force ambiguous records into confident categories. It may use different implicit criteria for different batches. It may summarize rather than preserve. It may give plausible rationales whose relation to the record is unclear. It may also expose useful structure that a human taxonomy missed.

The problem is not simply that model-assisted sorting is unreliable. The sharper problem is that unstructured model-assisted sorting is hard to inspect. Without a declared intent, fixed budget, frozen plan, required overflow space, and saved rationale trail, the analyst cannot easily tell what kind of semantic compression has occurred.

This paper proposes auditable semantic partitioning as a method for making that compression visible. The method separates the work into declared stages. The analyst supplies the dataset, sorting intent, field projection, model, and positive bucket count. The model performs a preflight budget judgment and proposes a bucket plan. The runtime enforces the requested number of positive buckets plus a required overflow bucket. Once generated, the bucket plan is frozen. Assignment then occurs against that locked plan, and the system records assignments, confidence scores, rationales, review flags, run artifacts, and analyst review notes.

The governing rule is simple:

*The user defines the budget. The model defines the semantic partition inside that budget.
The runtime enforces structure and preserves the audit trail.*

This rule turns model-assisted categorization from an opaque convenience into a reviewable data-work practice. The method does not claim that model-generated categories are objective ontology. It treats them as situated artifacts: outputs produced by a specific model, on a specific dataset projection, under a specific intent and bucket budget. The useful object is not only the sorted dataset. The useful object is the whole run.

2. Background and Gap

The paper sits near several existing practices but is identical to none of them.

Human-authored taxonomies are useful when a domain already has stable categories or when a study requires consistent coding. Their weakness appears earlier in the research process, when the analyst is still discovering which distinctions matter. A premature taxonomy can overdetermine the data by forcing every record through a frame that may not fit.

Topic modeling and clustering provide tools for discovering structure in unlabelled text. Classical topic models, embedding-based clustering, and neural pipelines such as BERTopic can reveal latent themes and produce interpretable topic representations. However, the analyst often receives clusters whose relation to the research task requires additional interpretation. The cluster may be mathematically coherent without being operationally useful, and the system may not preserve a full account of why each item belongs where it does.

Human-in-the-loop and active-learning workflows address different problems. They improve labels by putting human judgment back into the loop and by selecting informative examples for annotation. Auditable semantic partitioning shares the commitment to human review, but it does not assume the goal is to train or refine a supervised classifier. It can be used before a settled label space exists, when the analyst needs to explore what a dataset appears to afford.

Embedding search addresses retrieval: what is close to this query, item, or representation?
Auditable semantic partitioning addresses a whole-run question: what partition does this model propose under these declared conditions, where does it refuse fit, and how do those outputs change when the conditions change?

The gap is therefore methodological and infrastructural. Analysts need a way to use language models for exploratory semantic compression without treating model fluency as category truth. The method developed here aims to preserve the usefulness of model-native categorization while making the conditions, artifacts, and uncertainties available for review.

3. Method and Evidence Boundary

This is a methods and data-work paper with implementation-grounded examples, not a finished empirical benchmark. Its claim is that local language models can be used as inspectable semantic-partitioning instruments when the analyst declares the interpretive pressure, the bucket budget is enforced, uncertain material is preserved, and review artifacts are retained.

The necessary evidence for the current claim is narrower than a full evaluation program: a clear workflow, declared method primitives, source-anchored artifact types, and explicit non-claims around truth, objectivity, and model cognition. Multi-dataset runs, baseline comparisons, human-audit protocols, error analysis, and reproducibility packages would support a later technical or empirical version. They are future validation for this submission, not default prerequisites.

4. Implementation Case

The paper draws on two linked anonymized local-first projects.

The first is a local-first workbench for sorting datasets into semantic buckets using a GGUF-backed language model, Rust backend, local HTTP API, and web dashboard. Its documented operator flow includes dataset preview, semantic preflight, bucket-count judgment, optional force override, bucket-plan generation, full sort execution, artifact writing, review, comparison, snapshotting, and export. Its architecture separates domain types and validation (`core`), orchestration (`pipeline`), model adapters (`llm`), artifact writing and projection (`storage`), local API routes (`server`), and dashboard review (`ui`).

The sorter treats text files, folders, JSON, JSONL, Markdown, and Parquet rows as possible datasets. For structured sources such as Parquet, selected fields are projected into model-facing text while row identity is preserved for later audit. The implemented run artifacts include `run_config.json`, `preflight.json`, `bucket_plan.json`, `dataset_projection.json` where applicable, `assignments.jsonl`, `assignment_summary.json`, `run_manifest.json`, `run_summary.md`, bucket exports, snapshots, and analyst state.

The second project generalizes the same design stance. It defines a model-native semantic compression framework in which an application sets a fixed bucket count and the target model builds a reusable bucket map within that constraint. Its bucket-map schema records fields such as

model, prompt version, bucket count, source manifest, creation timestamp, bucket labels, summaries, anchor items, assigned items, and unassigned items. It also insists that changing the model should create a new bucket map rather than silently reusing another model's partition as though it were universal.

Together, these projects supply more than illustrative metaphors. They define implemented or specified artifacts, workflow boundaries, and audit surfaces. The paper uses them as an implementation-oriented case for a broader method.

5. Method Primitives

Auditable semantic partitioning can be described through a small set of primitives.

Primitive	Role in the method	Audit value
Dataset	The items to be sorted, such as files, notes, rows, issues, or excerpts	Fixes what material was under analysis
Projection	The model-facing view of each item, especially for structured data	Shows what the model could and could not see
Sorting intent	The question asked of the dataset	Prevents category output from floating free of purpose
Model	The local or remote language model used for the run	Makes partitions model-specific artifacts
Bucket budget	The requested number of positive buckets	Declares semantic bandwidth before the model names buckets
Preflight	Model judgment about whether the budget and intent appear suitable	Records objections before plan generation
Frozen plan	Bucket names, definitions, criteria, anchors, and overflow definition	Prevents category drift during assignment
Assignment	Per-item placement against the frozen plan	Produces the sorted data layer
Rationale	Model-facing explanation of a placement	Gives the human reviewer an inspectable deposition
Overflow/junk	Required space for weak-fit, ambiguous, mixed, or off-intent items	Preserves uncertainty rather than forcing certainty
Review state	Analyst verdicts, notes, watchlists, snapshots, and comparisons	Keeps human interpretation attached to the run

The most important primitive is the bucket budget. A bucket count is not a user-interface convenience. It is the semantic bandwidth of the run. If the analyst asks for three positive buckets, the model has only three positive distinctions available. If the analyst asks for twelve, the model has more resolution but also more opportunity to fragment. The method makes this pressure explicit rather than allowing the model to improvise an unbounded taxonomy.

6. Workflow

The workflow begins with dataset normalization. Text-native records can be used directly. Structured records require a projection step that selects which fields are visible and how they become model-facing text. This matters because a poor projection can destroy semantic signal before the model sees the record.

The analyst then declares a sorting intent. The intent might be topic, code function, operational failure mode, reasoning pattern, policy risk, classroom use case, or a custom instruction. Changing the intent changes what structure is visible. A dataset of software issues, for example, can be sorted by affected component, user pain, fix difficulty, reproducibility, security risk, or pedagogical value.

Before the plan is generated, the model runs preflight. It receives the intent, dataset sample, dataset shape metadata, and requested positive bucket count. It may judge the count acceptable, too low, too high, unclear, or weak-signal. This stage matters because it lets the model object without silently changing the count. If the operator forces continuation, the override is recorded.

The model then generates exactly the requested number of positive buckets plus the required overflow bucket. A useful plan includes bucket names, descriptions, criteria, anchors, an overflow definition, sorting-intent interpretation, bucket-shape rationale, signals noticed, weak-signal warnings, surprising groupings, and caution notes.

Only after the plan is frozen does assignment begin. The model classifies items against the frozen plan and returns a bucket ID, confidence score, rationale, and review flag. The runtime validates that bucket IDs are legal for the run and writes machine-facing and human-facing artifacts to disk.

Finally, the analyst reviews the run. Review may include accepting assignments, correcting them, flagging weak rationales, examining overflow contents, comparing snapshots, exporting bucket sets, or re-running the dataset under changed conditions. The method is complete only when the analyst can inspect not just the output, but the path by which the output was produced.

7. Bucket Genesis Modes

The source workbench distinguishes two bucket-generation modes: `blind_label` and `data_skim`.

In `blind_label` mode, the model names the bucket structure before seeing dataset content for bucket creation. The resulting plan expresses an expected ontology under the declared intent and bucket count. It asks: what categories does this model expect before inspecting the data?

In `data_skim` mode, the model sees dataset material before naming buckets. The resulting plan is more data-led. It asks: what categories does this dataset appear to suggest to this model under this intent and budget?

The comparison is methodologically useful. A category that appears in blind mode but not data-skim mode may reflect a model prior weakly supported by the dataset. A category that appears only after data skim may reflect a structure brought into view by the material. An item that lands in similar semantic regions across both modes may be a stable anchor. An item that moves may be context-dependent or weak-fit.

This does not reveal what the model "really thinks." It creates comparative evidence about output behavior under controlled conditions.

8. Overflow as Epistemic Restraint

Many classification systems treat miscellaneous or unassigned items as a defect. Auditable semantic partitioning treats overflow as part of the method.

The overflow bucket, called `junk` in one source system and `UNASSIGNED` in the other, holds weak-fit, ambiguous, mixed-content, off-intent, low-signal, or nonconforming items. Its purpose is to prevent the system from manufacturing certainty. If every item must belong somewhere, the system can conceal the boundary between a meaningful fit and a coerced one.

A high-overflow run is not automatically a failed run. It may indicate that the bucket budget is too small, the intent is vague, the projection is poor, the dataset is mixed, or the overflow subset deserves its own second pass. In this sense, overflow can become a refinement queue, ambiguity set, projection diagnostic, or out-of-distribution reservoir.

The ethical point is modest but important: uncertainty should remain visible when the categories do not cleanly cover the material.

9. Rationales as Audit Depositions

Assignment rationales are not transparent access to model internals. They are generated explanations. They can be shallow, circular, confabulatory, or excessively persuasive. For that reason, they should not be treated as ground truth.

They are still useful. A rationale records how the model justified a placement at the time of assignment. Human reviewers can inspect whether the rationale cites evidence present in the projected item, whether it matches the bucket criteria, whether it ignores obvious contrary signals, and whether similar items receive consistent reasoning. A poor rationale can expose forced fit. A rationale that cites absent evidence can expose hallucination or projection failure. A coherent rationale can support provisional acceptance.

Across repeated runs, rationales also help interpret disagreement. Two models may assign the same item differently because one emphasizes implementation risk while another emphasizes topic. The disagreement is then not merely an error count. It becomes evidence about what semantic pressure each model applies.

10. Semantic Triangulation

One run is a reading. Multiple runs are triangulation.

The same dataset can be sorted across conditions: different bucket counts, different sorting intents, `blind_label` versus `data_skim`, different local models, different field projections, full-dataset runs versus overflow-only refinement, and default prompts versus custom instructions.

Triangulation asks how a proposed structure behaves when those conditions change. Stable structures may be worth closer human inspection. Projection-sensitive structures reveal what information carries the signal. Model-specific structures reveal differences in model output behavior. Overflow growth may indicate that the categories are too narrow or that the dataset does not support the chosen intent.

The method therefore shifts the question from "did the model sort correctly?" to "what becomes visible, what refuses fit, and what changes under declared pressure?"

11. Future Validation Program

This paper does not report a finished empirical benchmark. It defines the future evaluation program needed before submission to a technical AI venue.

A fixed-taxonomy baseline would ask human authors to define categories first, then compare model and human assignment to those categories. This tests whether model-native bucket genesis adds useful structure beyond a human-authored label space.

An embedding-clustering baseline would embed records, cluster them, and label clusters using representative items or summaries. This tests whether the audit trail adds value beyond vector-space proximity.

A topic-modeling baseline would use methods such as BERTopic to discover topics and topic representations. This tests whether frozen bucket plans, overflow preservation, and rationales improve interpretability or downstream use.

A human-only baseline would ask reviewers to group the same records manually. This tests whether model-assisted sorting saves time, reveals missed structure, improves coverage, or creates new review burdens.

Measures should include reviewer time, inter-reviewer agreement, assignment acceptance rate, correction rate, overflow rate, review-flag precision, rationale usefulness, cluster/topic coherence, downstream retrieval or routing success, stability across repeated runs, sensitivity to bucket count, sensitivity to projection fields, and qualitative review of error patterns.

Candidate datasets should include mixed project notes, software issues, support records, research excerpts, Parquet rows, classroom questions, and deliberately heterogeneous folder corpora. The

key is not to select only tidy datasets where the method looks good. The method should be tested where ambiguity and weak signal are present.

12. Governance and Data Ethics

Local-first design matters because semantic sorting often touches private, messy, unpublished, or sensitive material. Project notes, drafts, internal logs, student work, support records, and research excerpts may not be appropriate for cloud upload. A local GGUF-backed workflow lets the analyst keep data on their own machine while still using model-assisted structure discovery.

Local does not automatically mean ethical. Analysts still need consent, security, provenance, and appropriate limits on use. But local-first architecture improves the possibility of operator control: inputs, configurations, outputs, overrides, bucket plans, assignments, review notes, snapshots, and exports can remain inspectable.

The method also requires strong interpretive humility. Model-native does not mean objectively correct. Stable does not mean true. Fluent does not mean justified. A bucket map is an artifact of a model, dataset projection, prompt, and budget. It should be cited and reused only with those conditions attached.

13. Risks and Limitations

The first risk is false precision. More buckets may make output appear more analytical without increasing actual reliability.

The second risk is bias. Model-native partitions may reproduce cultural, institutional, technical, or linguistic priors embedded in the model.

The third risk is rationale over-trust. Human reviewers may accept a persuasive explanation even when the assignment is weak.

The fourth risk is projection failure. If structured records are projected poorly, the model may never see the fields that carry meaning.

The fifth risk is repeated-run mystification. Agreement across runs is evidence of stability under tested conditions, not proof of objective semantic structure.

The sixth risk is local model variability. Smaller local models may produce weaker plans, less reliable assignment, or lower-quality rationales than larger cloud models. That variability should be measured, not hidden.

The seventh risk is workload displacement. Auditability creates review work. The method is valuable only if the artifacts improve human judgment rather than merely producing more material to inspect.

14. What This Paper Is Not Claiming

This paper does not claim that model-generated buckets are objective truth.

It does not claim direct access to hidden model cognition. Phrases such as "model-native" and "semantic priors" refer to observable outputs under declared conditions.

It does not claim that language-model sorting replaces topic modeling, clustering, supervised classification, or human annotation. It proposes a complementary workflow for settings where category discovery, local control, and auditability matter.

It does not claim that every high-overflow run is useful. Overflow becomes evidence only when preserved, inspected, and interpreted in relation to intent, projection, model, and bucket budget.

It does not claim that local-first inference is always better than cloud inference. It claims that local-first design is especially valuable when datasets are private, unpublished, sensitive, or experimental.

15. Conclusion

Auditable semantic partitioning reframes model-assisted categorization as controlled, inspectable semantic compression. The analyst declares a dataset, intent, projection, model, and bucket budget. The model proposes a partition within that constraint. The runtime freezes the plan, enforces legal assignments, preserves overflow, and records rationales and artifacts. The analyst reviews the run and compares it with alternatives.

The visible action is sorting. The deeper action is triangulation. The useful artifact is the whole run: intent, budget, model, projection, preflight, bucket plan, assignments, confidence scores, rationales, overflow, run artifacts, review notes, snapshots, and exports.

For Big Data & Society, the contribution is a data-work method for making model-assisted categorization more accountable to its own conditions. For future technical venues, the next step is empirical: benchmark datasets, baseline comparisons, human-audit protocols, error analysis, and reproducibility packages. The strongest version of the claim is not that the model knows the correct categories. It is that, under declared constraints, model-visible structure can become inspectable evidence rather than hidden convenience.